

Daniel Chang
C&S BIO 199
March 19, 2026
PI: Elisa Franco

Automated Python-Based Image Analysis Pipeline for Segmentation and Quantification of Biomolecular Condensates in Fluorescence Microscopy Z-Stacks

1. Abstract

This project aims to develop a fully automated, reproducible Python-based image analysis pipeline for fluorescence microscopy data to study biomolecular condensates in nuclear and cytoplasmic compartments. Using multi-channel Z-stack images from the JABr dataset, nuclei and condensates are segmented using pretrained Cellpose models implemented directly in Python. A preliminary comparison with a Python-native segmentation led to the selection of Cellpose for accurate and consistent object detection. Segmented masks are used to classify condensates as nuclear or cytoplasmic based on spatial overlap with nuclei. Quantitative object-level features are extracted across Z-slices, and the pipeline generates reproducible outputs including segmentation masks and structured summary tables. This framework enables scalable analysis of condensate behavior and supports future extensions such as time-lapse tracking.

2. Introduction

Biomolecular condensates are membrane-less cellular compartments that play important roles in processes such as gene expression and RNA regulation. Recent work from the Franco lab has shown that artificial RNA condensates can be engineered in living cells using programmable RNA nanostar motifs, which form distinct compartments in both the nucleus and cytoplasm through sequence-specific interactions [1].

Fluorescence microscopy enables visualization of these condensates; however, extracting quantitative and reproducible measurements from multi-channel Z-stack imaging data remains challenging. Manual and GUI-based approaches are difficult to scale and lack reproducibility, particularly when analyzing large datasets or comparing results across experimental conditions.

To address these limitations, this project develops an automated Python-based pipeline for condensate segmentation and quantification. The pipeline uses Cellpose models for nuclei and condensate segmentation, followed by compartment classification based on spatial overlap and

extraction of quantitative features. This framework enables consistent and scalable analysis of condensate behavior and facilitates investigation of how RNA design influences condensate localization and properties.

3. Data Description

3.1 Biological Context and Dataset

The data used in this project originates from experiments involving engineered RNA nanostar constructs designed to form synthetic biomolecular condensates in living cells. RNA nanostars are programmable RNA structures composed of multiple stem-loop arms connected by flexible spacers, enabling interactions that drive phase separation. These condensates serve as a model system for studying RNA-driven phase separation [1,2].

The naming convention for the RNA nanostar constructs reflects their structural design. In this dataset, JABr denotes a construct containing a 15-nt stem, a kissing-loop sequence (A), and the Broccoli fluorescent aptamer. These features influence interaction strength, condensate formation, and subcellular localization.

3.2 Dataset Organization and Baseline Reference

The dataset used in this project is part of a larger collection of fluorescence microscopy images and associated analysis outputs. It was previously analyzed using Cellpose-based segmentation and manual annotation, providing a reference baseline for validating the automated pipeline.

Relevant components include:

- Source Data: raw microscopy images
- Cut ROI: cropped regions of interest
- Masks: segmentation masks for nuclei and condensates
- Statistics / Partition Coefficient: processed quantitative outputs

These outputs serve as a reference for comparison with results generated by the pipeline.

3.3 Selected Sample

This project focuses on the sample:

- 20240516_JABr_40uMDFHBI_20-40_Sample2_5_MMStack_Pos0.ome.tif

This file is a multi-channel Z-stack fluorescence microscopy image of 55 slices containing two channels:

- Channel 1 (C1): nuclei signal
- Channel 2 (C2): Condensate signal (Broccoli-labeled RNA)

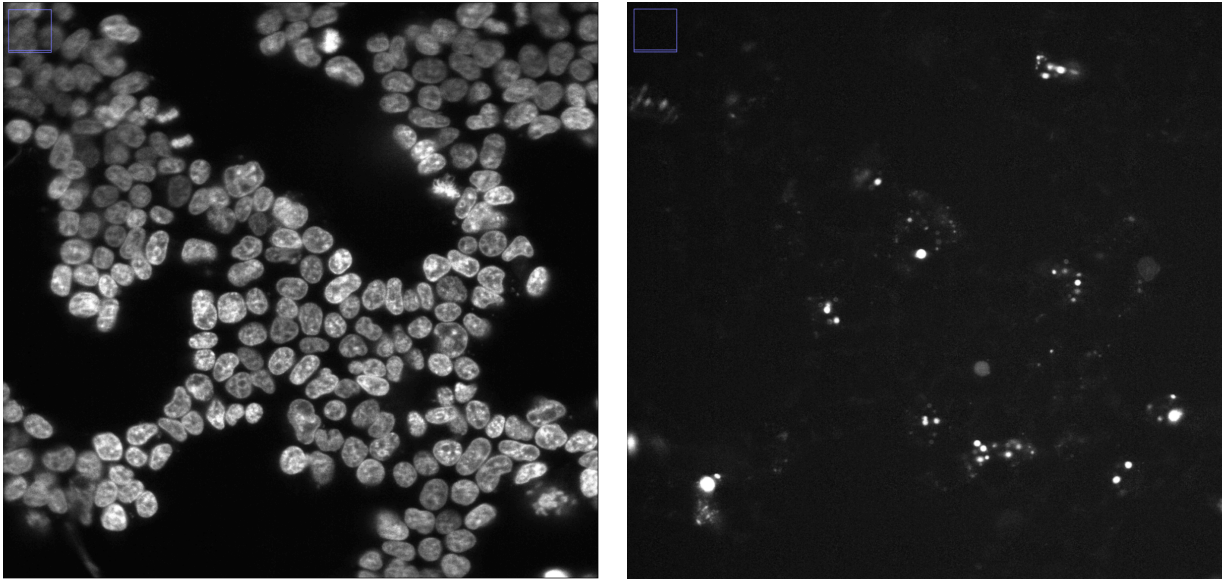


Figure 1. Channel Separation of Selected Sample

Channel 1 (left) shows nuclei signal, and Channel 2 (right) shows condensate signal. Images correspond to $z = 22$ of 55.

To enable independent processing, the channels were separated and stored as:

- C1-raw_stack_sample2_5.tif (nuclei channel)
- C2-raw_stack_sample2_5.tif (condensate channel)

3.4 Preprocessing and Region of Interest

To reduce computational complexity and enable focused validation, a cropped region of interest (ROI) was selected:

- roi_sample2_5_1.tif

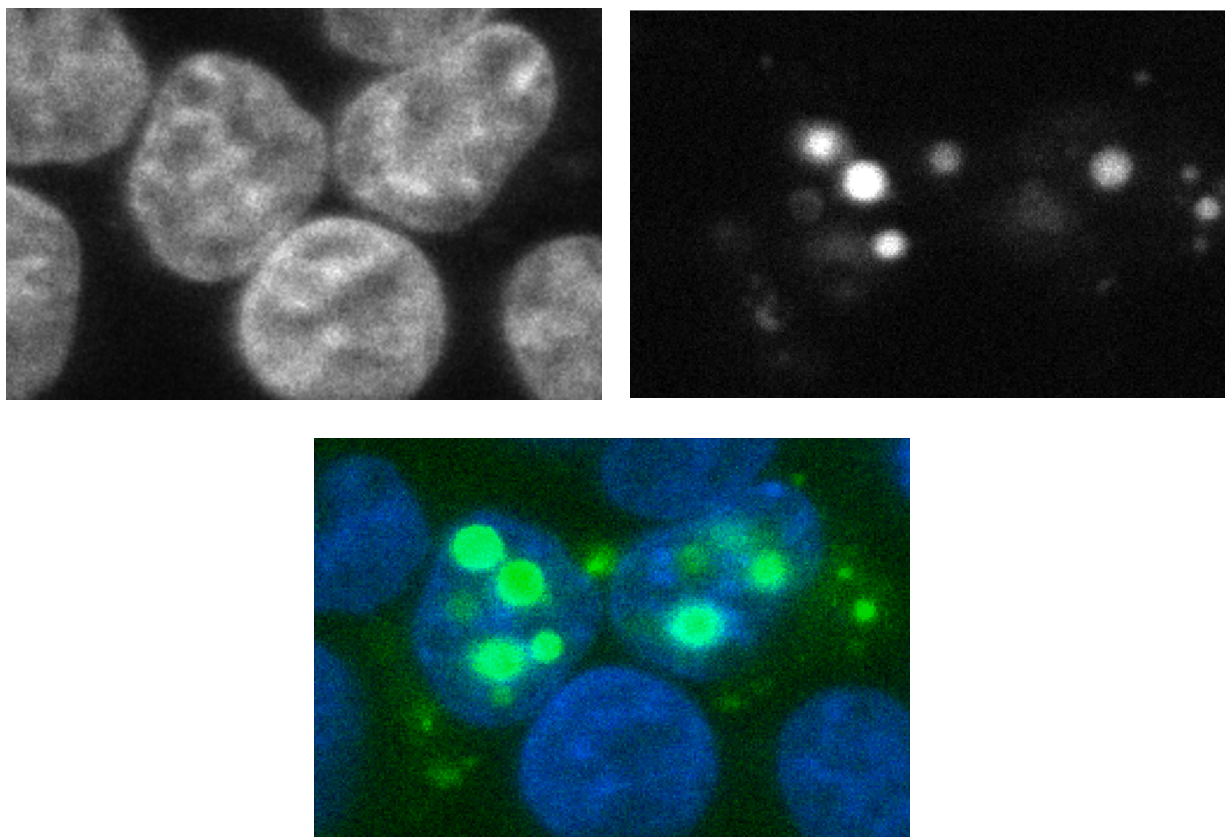


Figure 2. Region of Interest of Selected Sample

Top: nuclei (left) and condensates (right). Bottom: merged image. Images correspond to $z = 22$ of 55.

Corresponding masks with the ROI were available:

- Nuclei mask: sample_2_5_1_nuclei_mask.tif
- Condensates mask: sample_2_5_1_condensate_mask.tif



Figure 3. Masks Corresponding to Nuclei and Condensates of the Region of Interest
 Left: nuclei mask. Right: condensate mask. White indicates segmented objects. Images correspond to $z = 22$ of 55.

4. Pipeline Overview

The image analysis pipeline follows a structured workflow to ensure reproducibility and scalability.

Raw Data → Channel Separation → Nuclei Segmentation →
 Condensate Segmentation → Compartment Classification → Feature
 Extraction → Outputs (Masks, Graphs, and CSV Tables)

This workflow integrates segmentation with downstream quantitative analysis in a reproducible framework.

5. Methods

5.1 Channel Interpretation

The analysis was performed on the sample 20240516_JABr_40uMDFHBI_20-40_Sample2_5_MMStack_Pos0.ome.tif. The two channels were separated prior to analysis, with channel 1 used for nuclei segmentation and channel 2 used for condensate segmentation. The separated files were stored as C1-raw_stack_sample2_5.tif for nuclei and C2-raw_stack_sample2_5.tif for condensates.

5.2 Cellpose-Based Segmentation in Python

Segmentation was implemented directly in Python using Cellpose. Separate model instances were initialized for nuclei and condensate channels with GPU acceleration enabled. Each Z-slice was processed independently using `model.eval()` to generate instance segmentation masks. Both full-image and ROI-based analyses were performed to enable comparison with reference outputs.

5.3 Quantitative Feature Extraction

For each segmented object, quantitative measurements were extracted using `regionprops_table` from the `scikit-image` library. The extracted features included object label, area, centroid coordinates, and mean fluorescence intensity.

- Object label: unique identifier assigned to each segmented object within a Z-slice
- Area: number of pixels belonging to the object
- Centroid coordinates: spatial location of the object center, given as (centroid-0, centroid-1) corresponding to row (y) and column (x) positions
- Mean fluorescence intensity: average pixel intensity within the segmented object, computed from the raw fluorescence image

The measurements were computed independently for each Z-slice and aggregated into structured CSV tables for downstream analysis. These datasets were then used to generate summary graphs, including distributions of object area and mean fluorescence intensity, as well as object counts across Z-slices. Together, these outputs allow quantitative characterization of both nuclei and condensates, including size distributions, spatial localization, and intensity-based properties.

The pipeline also computes the nuclear partition coefficient, a summary measure of condensate enrichment. It is defined as the ratio of mean condensate intensity within nuclei to the mean dilute-phase intensity in the surrounding nuclear region outside condensates, using nuclei and condensate masks applied to the raw fluorescence images.

- Condensate intensity: pixels within nuclei $\cap$ condensates
- Dilute phase: pixels within nuclei but outside condensates
- Partition coefficient = condensate intensity / dilute-phase intensity

5.4 ROI-Based Analysis and Validation

The pipeline was applied to a cropped ROI for direct comparison with reference masks and dataset-derived statistics. This enabled both qualitative validation of segmentation accuracy and quantitative comparison using the partition coefficient.

5.5 Output Organization

The outputs were organized into structured files.

- Segmentation masks were saved as TIF stacks for both nuclei and condensates in separate folders
- Quantitative measurements were exported as CSV files in their respective folders.
 - Each row in the measurement tables corresponds to a single segmented object, with an associated Z-slice index

This organization facilitates integration with further analysis workflows, including statistical comparison and visualization.

6. Results

6.1 Output Organization

The pipeline generated segmentation masks for nuclei and condensates across the full Z-stack. Nuclei were segmented into well-defined contiguous regions, while condensates were identified as punctate structures of varying size and intensity.

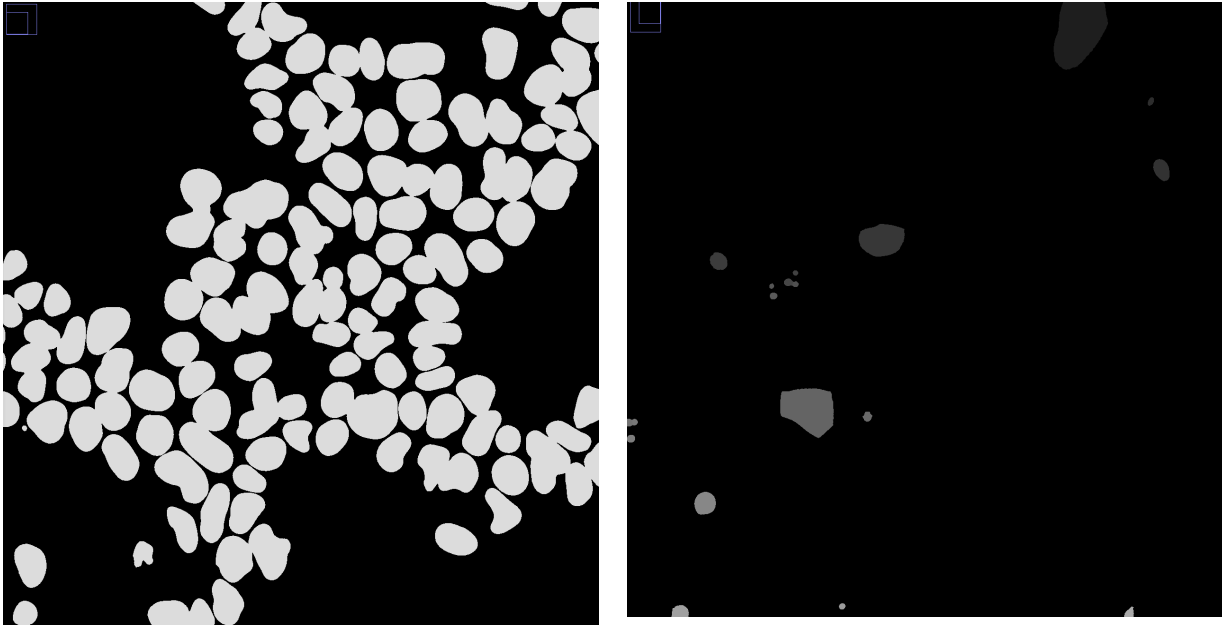


Figure 4. Segmentation masks for nuclei and condensates generated by the pipeline

Left: nuclei. Right: condensates. Images correspond to $z = 22$ of 55.

6.2 Quantitative Analysis of Segmented Objects

Quantitative features extracted from the segmentation masks were used to characterize the size, intensity, and spatial distribution of nuclei and condensates across the Z-stack. Histograms of object area show that condensates are predominantly small punctate structures, while nuclei exhibit larger and more uniform sizes.

Analysis of mean fluorescence intensity indicates greater variability among condensates compared to nuclei, while nuclei display a narrower intensity distribution. The number of detected objects per Z-slice further highlights differences in spatial organization, with nuclei counts remaining relatively stable and condensate counts showing greater fluctuation across slices.

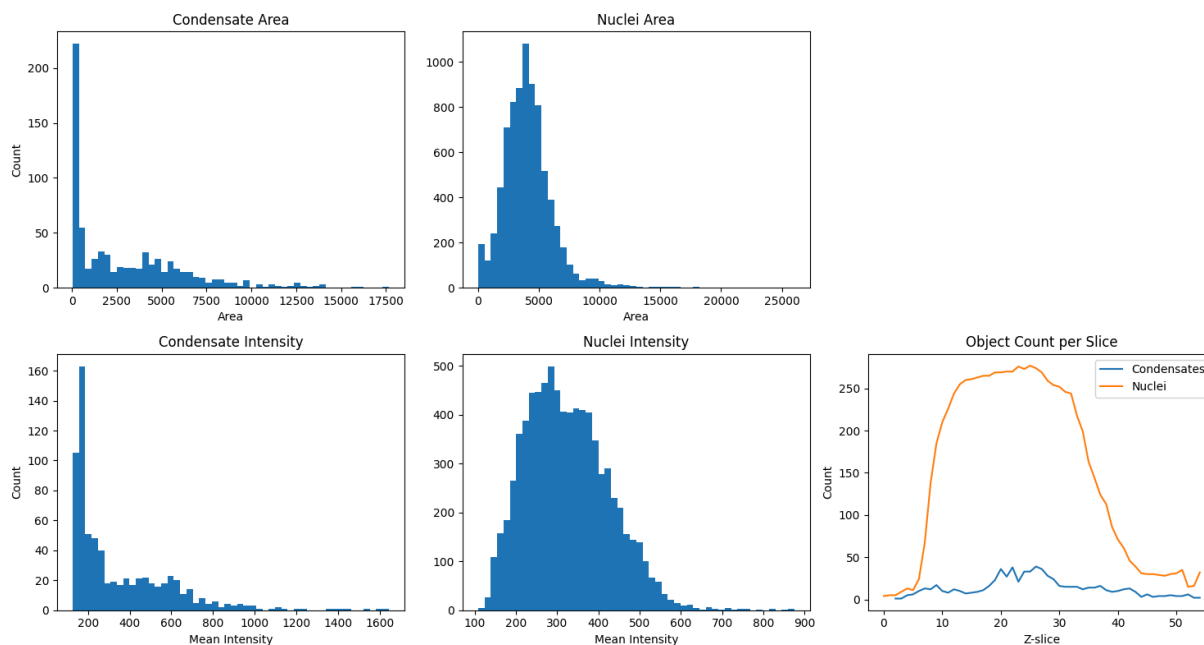


Figure 5. Quantitative analysis of segmented objects

Histograms of area and intensity, and object counts across Z-slices.

6.3 ROI Segmentation and Validation

To enable direct comparison with provided reference masks, the pipeline was applied to the cropped region of interest (ROI). Segmentation within the ROI produced masks consistent with the full-image results, with clearly defined nuclear regions and identifiable condensate structures suitable for validation.



Figure 6. ROI segmentation masks

Left: nuclei. Right: condensates. Images correspond to $z = 22$ of 55.

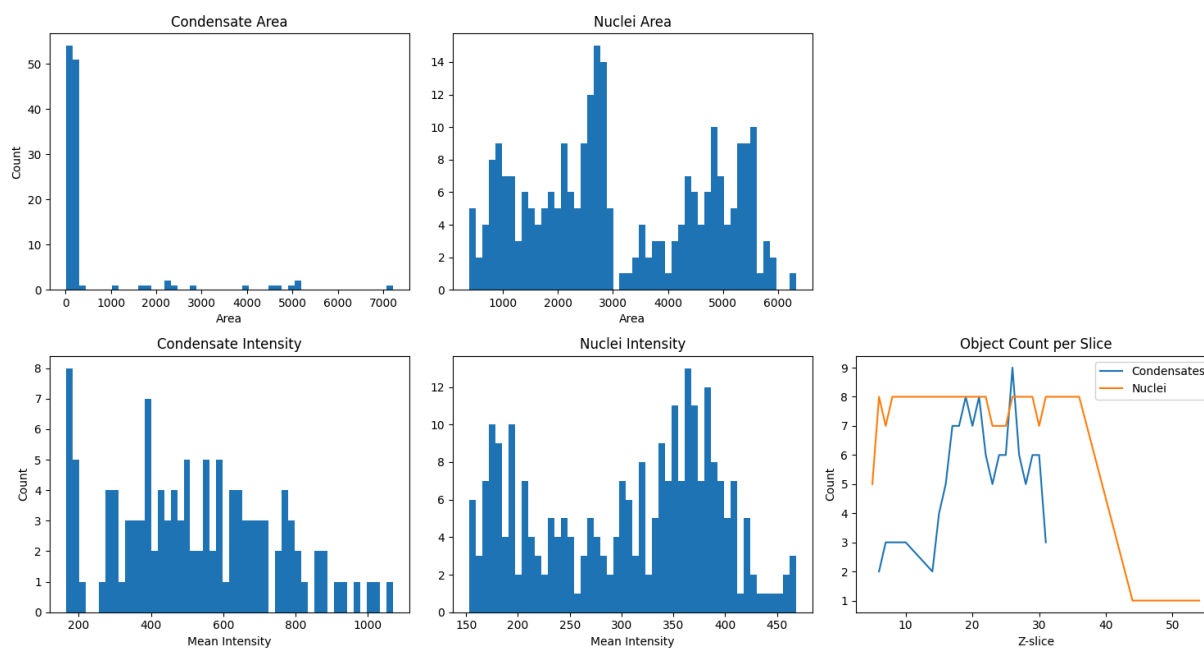


Figure 7. ROI quantitative analysis

Histograms of area and intensity, and object counts across Z-slices.

6.4 Partition Coefficient Analysis

The nuclear partition coefficient was computed to quantify condensate enrichment within nuclear regions. For the full image stack, the pipeline produced a condensate density of 249.17 and a dilute-phase density of 146.05, resulting in a partition coefficient of 1.71.

For the ROI, the pipeline produced a condensate density of 337.87 and a dilute-phase density of 165.31, resulting in a partition coefficient of 2.04. The ROI-based value is higher than the full-image result, consistent with reduced background influence within the cropped region. However, both values remain lower than the reference partition coefficient of 6.32.

7. Discussion

This project developed an automated Python-based pipeline for segmentation and quantitative analysis of biomolecular condensates from fluorescence microscopy Z-stacks. Overall, the results show that the pipeline can generate reproducible masks, extract object-level measurements, and compute a biologically meaningful summary metric through the nuclear partition coefficient.

Segmentation performance was stronger for nuclei than for condensates. Nuclei were detected consistently across the Z-stack and produced well-defined regions for downstream analysis. In contrast, condensates were more difficult to segment because they are smaller, more heterogeneous in intensity, and more sensitive to background signals. Although the pipeline captured the major punctate condensate structures, some low-intensity regions and possible artifacts were also included.

The quantitative results reflect these differences. Nuclei showed relatively stable object counts and more uniform size distributions, while condensates showed greater variation in count, area, and intensity across slices. This suggests that the pipeline is able to distinguish meaningful differences between the two object classes, while also showing that condensate measurements are more sensitive to segmentation quality.

The partition coefficient analysis further highlights both the usefulness and the limitations of the workflow. The ROI-based value (2.04) was higher than the full-image value (1.71), indicating improved local signal contrast when background is reduced. However, both values were lower than the reference value of 6.32. This suggests that small differences in condensate boundary detection can substantially affect downstream intensity-based measurements.

Overall, the pipeline provides a reproducible and scalable framework for condensate analysis, but the results also show that accurate quantitative interpretation depends heavily on segmentation quality. Future improvements could include additional parameter tuning,

post-processing to remove low-intensity artifacts, and formal comparison with reference masks using validation metrics such as Dice coefficient or IoU.

8. Reproducibility

The analysis pipeline was implemented entirely in Python using open-source libraries, including Cellpose for segmentation, NumPy and pandas for data processing, and scikit-image for feature extraction. All steps, including image loading, segmentation, quantitative feature extraction, and visualization, were executed through a single, reproducible workflow.

The pipeline operates on standardized input files, including multi-channel Z-stack TIFF images, and produces consistent outputs in the form of segmentation mask stacks (TIF) and quantitative measurement tables (CSV). File organization and naming conventions were structured to ensure clarity and ease of reuse.

GPU acceleration was used to improve computational efficiency during segmentation, enabling scalable analysis of large image stacks. The modular design of the pipeline allows it to be applied to additional datasets or extended with minimal modification.

A limitation of the pipeline is its dependence on available computational resources. While GPU acceleration substantially reduces runtime, execution on systems without a GPU (e.g., standard laptops) may be significantly slower, particularly for large Z-stack datasets. This may impact scalability and processing time in resource-constrained environments.

8.1 Computational Environment

The pipeline was developed and tested on two systems:

- MacBook Pro (Apple M1 Pro, 10-core CPU, 16 GB RAM)
- Desktop workstation (AMD Ryzen 7 7700 CPU, NVIDIA RTX 4080 GPU, 32 GB RAM)

GPU acceleration on the desktop (CUDA-enabled RTX 4080) significantly reduced runtime for segmentation tasks compared to Metal-based acceleration on the Apple M1 Pro laptop.

9. Conclusion

This project developed a reproducible Python-based pipeline for segmentation and quantitative analysis of biomolecular condensates from multi-channel fluorescence microscopy Z-stacks. The workflow generates segmentation masks, extracts quantitative object features, and computes partition coefficients to measure condensate enrichment within nuclei.

The results show that the pipeline performs well for nuclei and captures major condensate structures, while also revealing that condensate segmentation remains the main source of quantitative error. The difference between computed and reference partition coefficients demonstrates that visually reasonable masks do not always guarantee accurate downstream measurements.

Despite these limitations, the pipeline establishes a strong foundation for automated and scalable condensate image analysis. With further optimization and validation, it can support analysis of additional samples and future extensions such as comparative studies across constructs or time-lapse tracking.

10. Data Availability

The microscopy data, ROI files, reference masks, and selected analysis outputs used in this project are available in a shared Google Drive folder:

https://drive.google.com/drive/folders/1ID07uZFhcz3lQGn6lbLoFc47y6NH3_IG?usp=drive_link

11. References

[1] Li, S., Kim, Y., Wang, K., Payson, E. J., Tang, A. A., Villalba Nieto, M., Osmanovic, D., Yang, M., Dilao, D., Bermudez, A., Xiao, W., Li, M. M. H., Lin, N. Y. C., Plath, K., Black, D. L., & Franco, E. (2026). Programmable artificial RNA condensates in mammalian cells. bioRxiv. <https://doi.org/10.64898/2026.01.28.702393>

[2] Banani, S. F., Lee, H. O., Hyman, A. A., & Rosen, M. K. Biomolecular condensates: organizers of cellular biochemistry. *Cell*, 2017. https://pmc.ncbi.nlm.nih.gov/articles/PMC7434221/?utm_source

Appendix A: Pipeline Implementation

https://github.com/Dauniel/research_lab

```
from pathlib import Path

import numpy as np
import pandas as pd
import tifffile as tiff
from cellpose import models
from skimage.measure import regionprops_table

import torch
```

```

from cellpose import core
import matplotlib.pyplot as plt

# setting up paths

BASE_DIR = Path.cwd()
DATA_DIR = BASE_DIR / "data"
OUTPUT_DIR = BASE_DIR / "outputs" / "cellpose_python"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

COND_PATH = DATA_DIR / "raw_mask_condensates" /
"C2-raw_stack_sample2_5.tif"
NUC_PATH = DATA_DIR / "raw_mask_nuclei" / "C1-raw_stack_sample2_5.tif"

# checking input files

print("Condensate file exists: ", COND_PATH.exists())
print("Nuclei file exists: ", NUC_PATH.exists())

# loading image stacks

cond_stack = tiff.imread(COND_PATH)
nuc_stack = tiff.imread(NUC_PATH)

# checking stack information

print("Condensates shape:", cond_stack.shape)
print("Nuclei shape:", nuc_stack.shape)
print("Condensates dtype:", cond_stack.dtype)
print("Nuclei dtype:", nuc_stack.dtype)

# loading cellpose models

cond_model = models.CellposeModel(gpu=True)
nuc_model = models.CellposeModel(gpu=True)

# checking gpu status

print("GPU working:", core.use_gpu())
print("CUDA available:", torch.cuda.is_available())
print("GPU count:", torch.cuda.device_count())
print("Current device:", torch.cuda.current_device())
print("GPU name:", torch.cuda.get_device_name(torch.cuda.current_device()))

```

```

# segmentation function

def run_cellpose_stack(stack, model, object_name="objects", diameter=None):
    mask_stack = []
    measurements = []

    for z in range(stack.shape[0]):
        img = stack[z]

        # running cellpose on one slice

        masks, flows, styles = model.eval(
            img,
            diameter=diameter,
        )

        mask_stack.append(masks.astype(np.int32))

        # extracting object measurements

        props = regionprops_table(
            masks,
            intensity_image=img,
            properties=["label", "area", "centroid", "mean_intensity"]
        )

        df = pd.DataFrame(props)

        # adding slice index

        if not df.empty:
            df["z"] = z
        else:
            df = pd.DataFrame(columns=["label", "area", "centroid-0",
            "centroid-1", "mean_intensity", "z"])

        measurements.append(df)

        print(f"{object_name} slice {z}: {masks.max()} objects")

    # combining outputs across slices

```

```

mask_stack = np.stack(mask_stack)
measurements_df = pd.concat(measurements, ignore_index=True)

return mask_stack, measurements_df

# running segmentation for condensates

cond_masks, cond_df = run_cellpose_stack(
    cond_stack,
    cond_model,
    object_name="condensates",
    diameter=None
)

# running segmentation for nuclei

nuc_masks, nuc_df = run_cellpose_stack(
    nuc_stack,
    nuc_model,
    object_name="nuclei",
    diameter=None
)

# checking segmentation outputs

print("Condensate mask stack shape:", cond_masks.shape)
print("Nuclei mask stack shape:", nuc_masks.shape)

print(cond_df.head())
print(nuc_df.head())

# partition coefficient calculation

cond_pixels_all = []
dilute_pixels_all = []

for z in range(cond_stack.shape[0]):
    cond_mask = cond_masks[z] > 0
    nuc_mask = nuc_masks[z] > 0
    img = cond_stack[z]

    # getting condensate and dilute phase pixels inside nuclei

```

```

cond_pixels = img[cond_mask & nuc_mask]
dilute_pixels = img[nuc_mask & (~cond_mask)]

if cond_pixels.size > 0:
    cond_pixels_all.append(cond_pixels)

if dilute_pixels.size > 0:
    dilute_pixels_all.append(dilute_pixels)

# computing mean densities and partition coefficient

cond_density = np.mean(np.concatenate(cond_pixels_all))
dilute_density = np.mean(np.concatenate(dilute_pixels_all))
partition_coefficient = cond_density / dilute_density

# saving masks and measurement tables

tiff.imwrite(OUTPUT_DIR / "ROI_condensate_masks.tif",
cond_masks.astype(np.int32))
tiff.imwrite(OUTPUT_DIR / "ROI_nuclei_masks.tif",
nuc_masks.astype(np.int32))

cond_df.to_csv(OUTPUT_DIR / "ROI_condensate_measurements.csv", index=False)
nuc_df.to_csv(OUTPUT_DIR / "ROI_nuclei_measurements.csv", index=False)

print("Saved to:", OUTPUT_DIR)

# plotting summary figures

fig, axs = plt.subplots(2, 3, figsize=(15, 8))

axs[0,0].hist(cond_df["area"], bins=50)
axs[0,0].set_title("Condensate Area")
axs[0,0].set_xlabel("Area")
axs[0,0].set_ylabel("Count")

axs[0,1].hist(nuc_df["area"], bins=50)
axs[0,1].set_title("Nuclei Area")
axs[0,1].set_xlabel("Area")

axs[0,2].axis("off")

axs[1,0].hist(cond_df["mean_intensity"], bins=50)

```



```
axs[1,0].set_title("Condensate Intensity")
axs[1,0].set_xlabel("Mean Intensity")
axs[1,0].set_ylabel("Count")

axs[1,1].hist(nuc_df["mean_intensity"], bins=50)
axs[1,1].set_title("Nuclei Intensity")
axs[1,1].set_xlabel("Mean Intensity")

# counting objects per slice

cond_counts = cond_df.groupby("z")["label"].count()
nuc_counts = nuc_df.groupby("z")["label"].count()

axs[1,2].plot(cond_counts.index, cond_counts.values, label="Condensates")
axs[1,2].plot(nuc_counts.index, nuc_counts.values, label="Nuclei")
axs[1,2].set_title("Object Count per Slice")
axs[1,2].set_xlabel("Z-slice")
axs[1,2].set_ylabel("Count")
axs[1,2].legend()

plt.tight_layout()
plt.show()
```